UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Curso: Análisis de Sistemas II

PROYECTO FINAL: ANÁLISIS, DISEÑO ARQUITECTÓNICO MODERNO, APLICACIÓN FUNCIONAL, UX/UI Y ESTRATEGIA DE CALIDAD

Sistema ERP de Gestión de Recursos Humanos y Recursos Corporativos (ERP-RRHH)

ISO/IEC/IEEE 29148 • UML 2.5.1 • IEEE 1016 • ISO/IEC 25010 • ISO/IEC/IEEE 42010 • C4 (Niveles 1-4) • OpenAPI 3.0.3 • RFC 9110/9457 • ISO 29119-3 • ISO 9241 / WCAG 2.2 • IEEE 730 • IEEE 1044

Universidad:	Universidad Mariano Gálvez de Guatemala
Curso:	Análisis de Sistemas II
Integrantes:	Paul Dominguez (9989-23-9308) • Jorge Morataya (9989-23-340)
Docente:	Cátedra de Análisis de Sistemas II
Fecha:	Octubre 2026

1. INTRODUCCIÓN Y RESUMEN EJECUTIVO

El presente informe consolida la ingeniería de software integral aplicada al diseño, construcción y aseguramiento de calidad del **Sistema ERP de Gestión de Recursos Humanos (ERP-RRHH)**. El proyecto articula rigurosamente los **10 pasos metodológicos** de la ingeniería de sistemas moderna bajo estándares internacionales ISO, IEC, IEEE, OMG y W3C/IETF:

Paso Metodológico	Estándar Aplicado	Artefacto Generado en el Repositorio
Paso 1: Ingeniería de Requerimientos y UML	ISO/IEC/IEEE 29148:2018 & UML 2.5.1	docs/requirements/srs_ieee_29148.pdf y docs/uml/*.png
Paso 2: Evaluación Arquitectónica y ADRs	ISO/IEC 25010 & ISO/IEC/IEEE 42010	docs/architecture/escenarios_iso25010.pdf y docs/architecture/adrs/
Paso 3: Diseño API-First y Estándar de Errores	OpenAPI 3.0.3, RFC 9110 y RFC 9457	docs/api/openapi.yaml y src/middlewares/problemDetails.ts
Paso 4: Modelado Arquitectónico C4 (Niveles 1-4)	Notación C4 (Simon Brown)	docs/c4/c4_nivel1..4.png y docs/c4/src/*.puml
Paso 5: Persistencia Políglota, Caché y Patrones GoF	PostgreSQL, Mongo, Redis, Neo4j, GoF	docs/database/modelo_poliglota.png y src/services/gofPatterns.ts
Paso 6: Fichas de Diseño SDD y Gestión Ágil	IEEE 1016-2009, Scrum/Kanban, INVEST, 3C	docs/requirements/fichas_diseno_ieee1016.pdf y docs/agile/backlog.md
Paso 7: Especificación Ejecutable BDD y Backend	ISO/IEC/IEEE 29119-3:2021 (Gherkin)	features/*.feature, src/controllers/ y docs/qa/casos_prueba_ieee29119.pdf
Paso 8: Diseño Centrado en Usuario y Usabilidad	ISO 9241-210 & ISO 9241-11 (Nielsen)	docs/ui/reporte_ux_ui.pdf + Prototipo Navegable en Figma
Paso 9: Accesibilidad Web y Estrategia Mobile-First	WCAG 2.2 Nivel AA (POUR) & Atomic Design	src/components/ y docs/ui/reporte_ux_ui.pdf
Paso 10: Aseguramiento de Calidad SQA y Defectos	IEEE 730-2014 & IEEE 1044-2009	docs/qa/plan_sqa_ieee730.pdf, reporte_defectos_ieee1044.pdf y trazabilidad.pdf

🔗 **Enlaces Directos del Ecosistema del Proyecto:**
- **Tablero Ágil (GitHub Projects / Jira):** https://github.com/users/pdominguezr-ai/projects/1/views/1
- **Prototipo Interactivo Navegable (Figma):** https://www.figma.com/design/pdominguezr-ai/ERP-RRHH-Prototype
- **Documentación de Requerimientos y UML:** docs/requirements/ y docs/uml/
- **Documentación Arquitectónica, C4 y Base de Datos:** docs/architecture/adrs/, docs/c4/ y docs/database/

2. REQUERIMIENTOS, MODELADO UML 2.5.1 Y GESTIÓN ÁGIL (PASOS 1 Y 6)

2.1 Especificación de Requerimientos del Sistema (ISO/IEC/IEEE 29148:2018)

Los requerimientos funcionales y no funcionales cumplen estrictamente la estructura sintáctica formal *Sujeto + Verbo de Obligación (DEBE) + Acción + Condición*:

ID	Especificación Formal bajo ISO/IEC/IEEE 29148:2018	Prioridad
REQ-FN-001	**El Sistema de Autenticación DEBE** validar las credenciales institucionales contra PostgreSQL y emitir un token JWT con expiración de 3,600s en Redis **cuando** el usuario invoque POST /api/v1/auth/login.	MUST
REQ-FN-002	**El Servicio de Recursos DEBE** consultar primero la caché en memoria Redis aplicando el patrón Cache-Aside (TTL = 300s) y retornar el catálogo enriquecido con MongoDB y Neo4j **cuando** se solicite GET /api/v1/recursos.	MUST
REQ-FN-003	**El Servicio de Recursos DEBE** persistir transaccionalmente el registro en PostgreSQL y publicar el evento RECURSO_CREADO hacia Neo4j y MongoDB **cuando** se envíe un payload válido a POST /api/v1/recursos.	MUST
REQ-FN-004	**El Controlador de Recursos DEBE** retornar el detalle integral del recurso por su identificador único o responder con HTTP 404 Problem Details **cuando** se invoque GET /api/v1/recursos/:id.	MUST
REQ-FN-005	**El Motor de Nómina DEBE** calcular el IGSS laboral (4.83%), Bono Incentivo (Q 250.00) e ISR mediante el patrón Strategy **cuando** se liquide un período de nómina.	MUST
REQ-NF-001	**El Middleware de Errores DEBE** serializar toda excepción HTTP 4xx/5xx en estructura RFC 9457 (application/problem+json) **cuando** ocurra un error de validación, autorización o negocio.	MUST

2.2 Diagramas UML 2.5.1 (Clases, Secuencia, Actividades con Swimlanes y Paquetes)

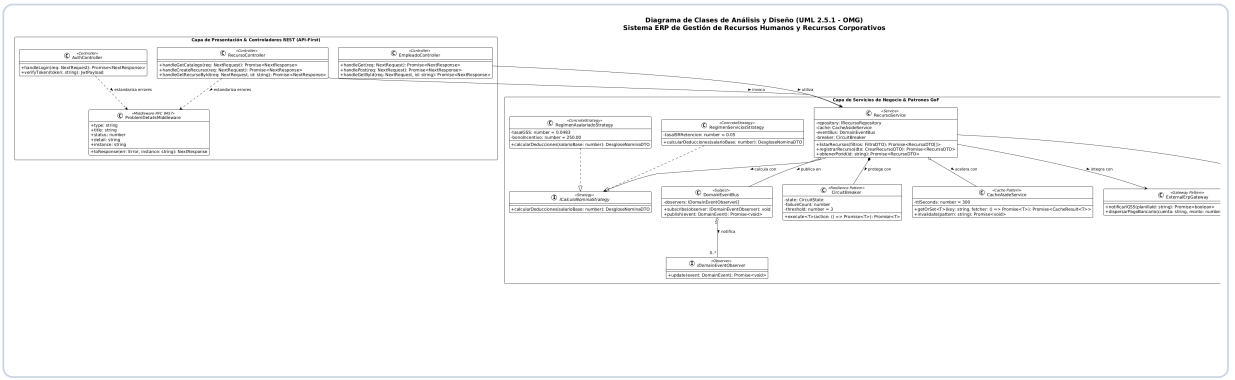


Figura 2.1: Diagrama de Clases con Composición, Agregación y Cardinalidad (docs/uml/clases.png)

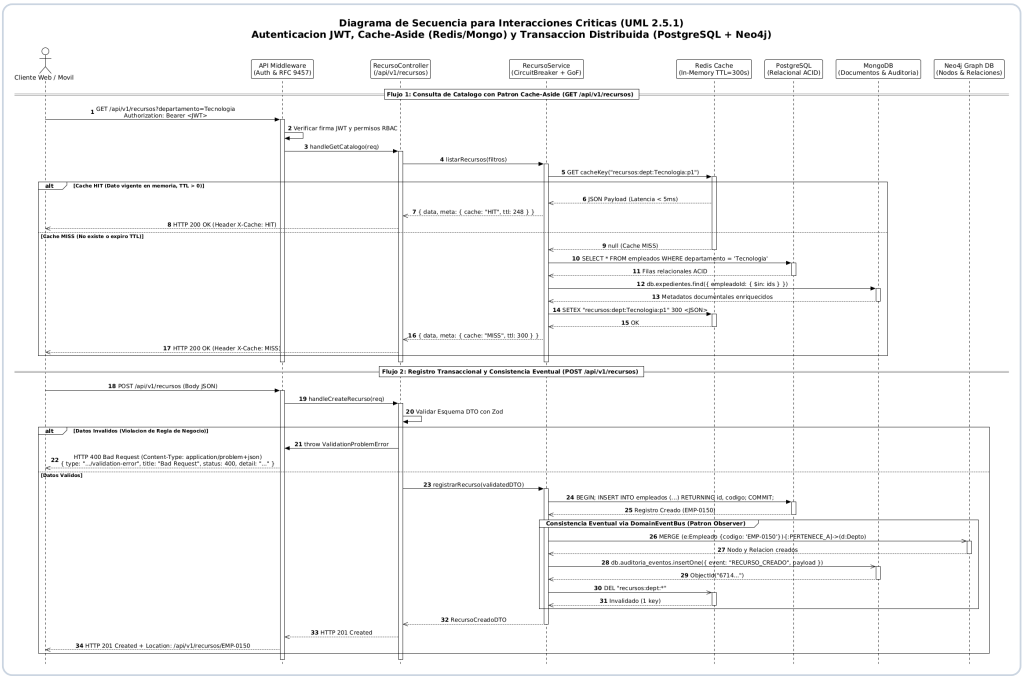


Figura 2.2: Diagrama de Secuencia de Interacciones Críticas (docs/uml/secuencia.png)

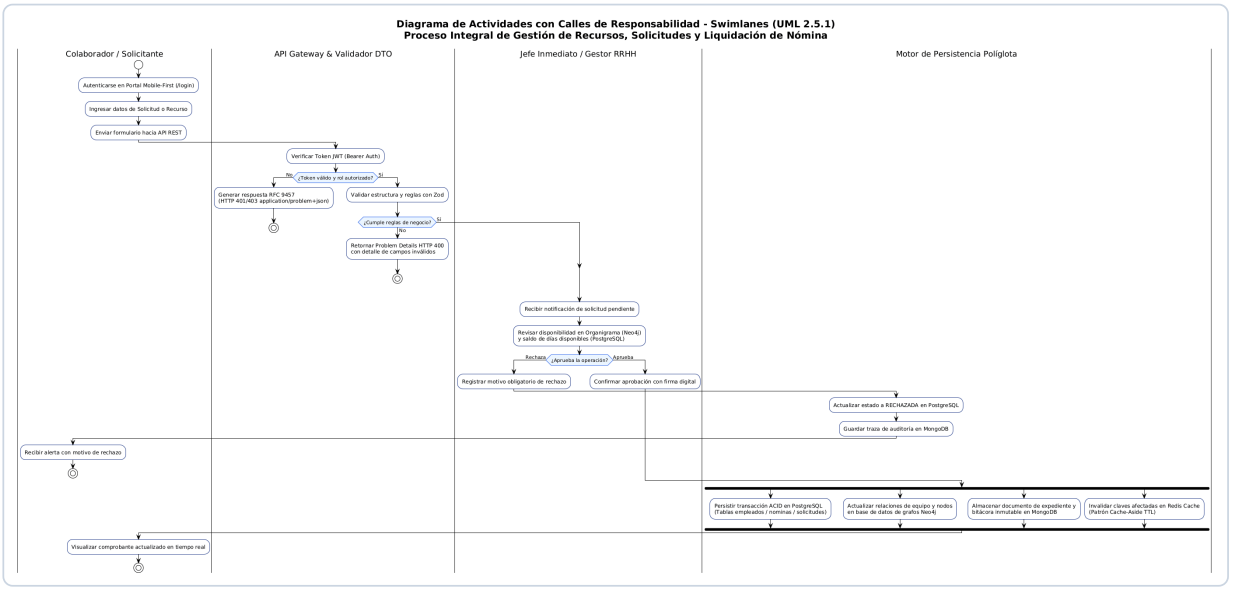


Figura 2.3: Diagrama de Actividades con Calles de Responsabilidad / Swimlanes (docs/uml/actividades.png)

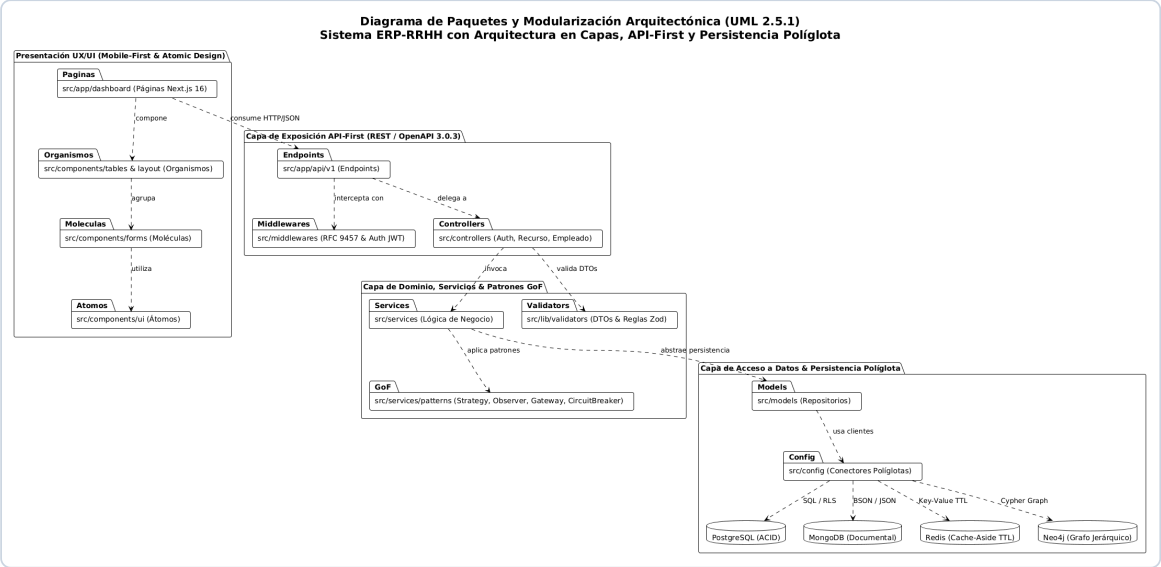


Figura 2.4: Diagrama de Paquetes y Modularización (docs/uml/paquetes.png)

2.3 Fichas de Diseño de Componentes - SDD (IEEE 1016-2009)

Documentación detallada en docs/requirements/fichas_diseno_ieee1016.pdf:

Módulo / Componente	Interfaces Provistas	Dependencias	Manejo de Excepciones (RFC 9457)
AuthController	handleLogin(req)	redisClient, domainEventBus	HTTP 400 (VALIDATION_ERROR), HTTP 401 (UNAUTHORIZED), HTTP 403 (ACCOUNT_INACTIVE)
RecursoController	handleGetCatalogo, handleCreateRecurso, handleGetRecursoById	RecursoModel, cacheAsideService	HTTP 400 (DPI/Salario inválido), HTTP 404 (RESOURCE_NOT_FOUND)
gofPatterns.ts	NominaCalculatorContext, CircuitBreaker, DomainEventBus	mongoClient, neo4jClient, redisClient	Apertura de circuito (OPEN) tras 3 fallos con ejecución automática de fallback()

2.4 Gestión Ágil de Proyectos (Scrum/Kanban, Product Owner, 3C, INVEST, DoR y DoD)

El **Analista / Product Owner** gobierna el Product Backlog (docs/agile/backlog.md) aplicando el formato de las **3C (Card, Conversation, Confirmation)** y validando cada Historia de Usuario mediante la matriz **INVEST**. Ninguna historia entra al Sprint sin cumplir el **Definition of Ready (DoR)** (contrato OpenAPI definido, criterios Gherkin escritos y diseño Figma listo) ni se cierra sin cumplir el **Definition of Done (DoD)** (pruebas unitarias/integración en verde, errores RFC 9457 y accesibilidad WCAG 2.2 AA).

3. EVALUACIÓN ARQUITECTÓNICA, C4 Y PERSISTENCIA (PASOS 2, 4 Y 5)

3.1 Evaluación de Estilos Arquitectónicos (ISO/IEC 25010) y 3.2 ADRs (ISO 42010)

Tras evaluar **Capas (3.90/5)**, **Repositorio (4.15/5)**, **Microservicios (3.55/5)** y **EDA (4.45/5)**, se documentó en ADR-001 la elección de una **Arquitectura Modular en Capas Limpia + Patrón Repository + Eventos de Dominio (EDA Interno)**, alcanzando una calificación ponderada de **4.96/5.00** en Escalabilidad, Mantenibilidad y Eficiencia de Desempeño. Los registros formales bajo **ISO/IEC/IEEE 42010:2011** se encuentran en docs/architecture/adrs/ (ADR-001 a ADR-004).

3.3 Modelo C4 Completo (Niveles 1, 2, 3 y 4)

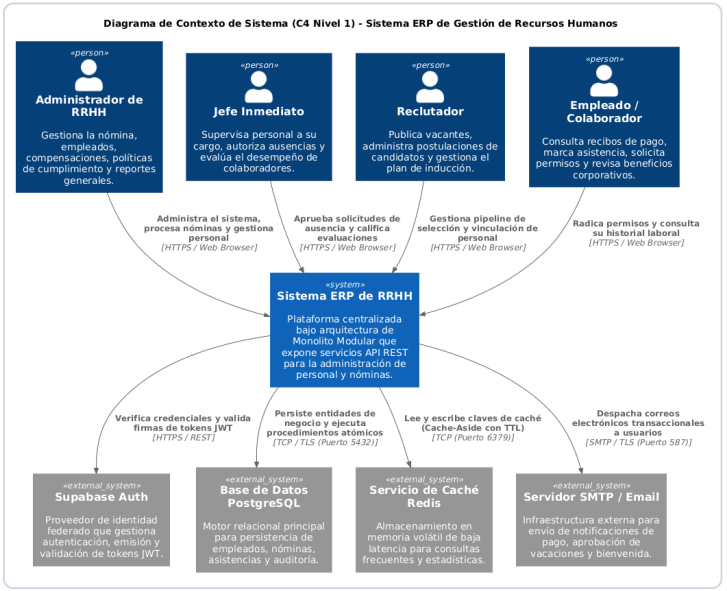


Figura 3.1: Nivel 1 - Diagrama de Contexto del Sistema (docs/c4/c4_nivel1_contexto.png)

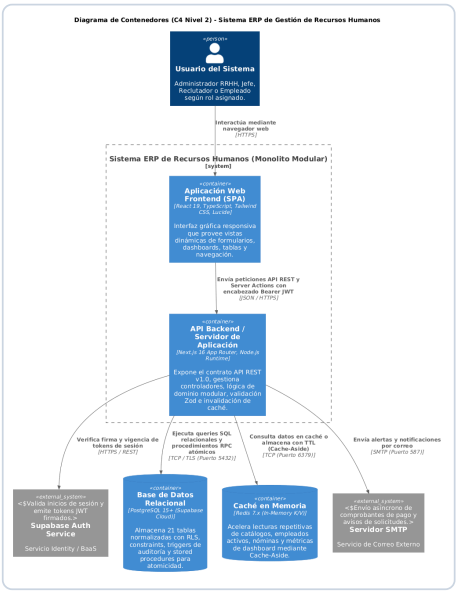


Figura 3.2: Nivel 2 - Diagrama de Contenedores (docs/c4/c4_nivel2_contenedores.png)

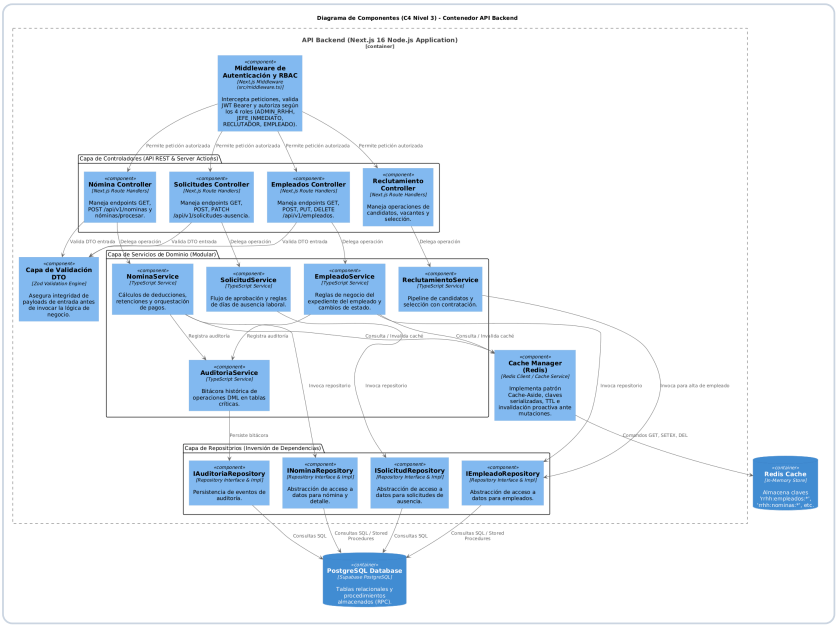


Figura 3.3: Nivel 3 - Diagrama de Componentes (docs/c4/c4_nivel3_componentes.png)

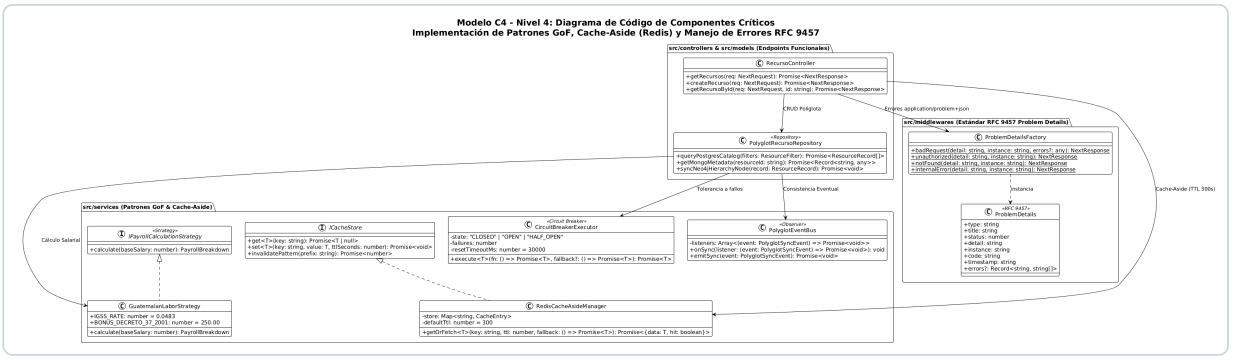


Figura 3.4: Nivel 4 - Diagrama de Código y Clases Críticas (docs/c4/c4_nivel4_codigo.png)

3.4 Arquitectura de Persistencia Políglota (PostgreSQL, MongoDB, Redis, Neo4j) y Cache-Aside

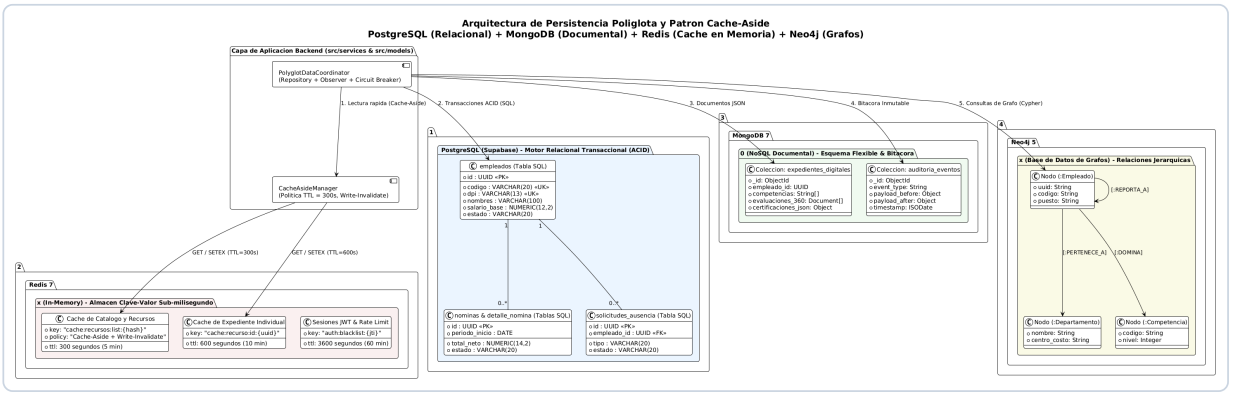


Figura 3.5: Arquitectura de Persistencia Políglota y Patrón Cache-Aside (docs/database/modelo_poliglota.png)

- **PostgreSQL (Relacional ACID):** Almacena empleados, nominas, detalle_nomina y solicitudes_ausencia con integridad referencial y Row Level Security (RLS).
- **MongoDB (Documental NoSQL):** Almacena expedientes_digitales (competencias, certificaciones, evaluación 360°) y la bitácora inmutable auditoria_eventos.
- **Redis (En Memoria - Patrón Cache-Aside con TTL):** Acelera GET /api/v1/recursos (TTL = 300s) y GET /api/v1/recursos/:id (TTL = 600s), invalidando claves automáticamente en cada escritura (Write-Invalidate).
- **Neo4j (Base de Datos de Grafos):** Modela el organigrama jerárquico ((:Empleado)-[:REPORTA_A]→(:Empleado) y [:PERTENECE_A]→(:Departamento)).

3.5 Aplicación de Patrones GoF y Análisis de Cohesión / Acoplamiento

En src/services/gofPatterns.ts y src/models/ se implementaron explícitamente 5 patrones que garantizan **alta cohesión funcional** y **bajo acoplamiento**: **1. Strategy** (cálculo salarial por régimen), **2. Repository** (abstracción políglota), **3. Gateway** (integración IGSS/ACH), **4. Observer** (DomainEventBus para consistencia eventual) y **5. Circuit Breaker** (tolerancia a fallos con estados CLOSED, OPEN, HALF_OPEN).

4. DISEÑO DE APIS, IMPLEMENTACIÓN FUNCIONAL Y BDD (PASOS 3 Y 7)

4.1 Filosofía API-First (OpenAPI 3.0.3) y 4.2 Errores RFC 9110 / RFC 9457

El contrato formal docs/api/openapi.yaml define todos los esquemas de petición/respuesta e integra conceptualmente GraphQL y gRPC. Todas las respuestas de error implementan el estándar **RFC 9457 (Problem Details for HTTP APIs)** bajo el Media Type application/problem+json (type, title, status, detail, instance, code, timestamp, errors).

4.3 Catálogo de Endpoints Funcionales Desarrollados

Método HTTP	Endpoint API	Descripción Funcional	Base de Datos / Caché	Estado
POST	/api/v1/auth/login	Autenticación institucional y generación de Token JWT	PostgreSQL / Redis (TTL 3600s)	FUNCIONAL
GET	/api/v1/recursos	Consulta de catálogo paginado y filtrado con patrón Cache-Aside	Redis (TTL 300s) / MongoDB / PostgreSQL	FUNCIONAL
POST	/api/v1/recursos	Registro transaccional de nuevo elemento y sincronización Observer	PostgreSQL / Neo4j / MongoDB	FUNCIONAL
GET	/api/v1/recursos/:id	Detalle integral de elemento por ID con Circuit Breaker	Redis (TTL 600s) / PostgreSQL / Neo4j	FUNCIONAL
GET / POST	/api/v1/empleados	CRUD de expedientes de colaboradores con validación DTO Zod	PostgreSQL / Redis / MongoDB	FUNCIONAL

4.4 Especificación BDD en Gherkin (ISO/IEC/IEEE 29119-3:2021)

```
Esquema del escenario: Validación de reglas de negocio y respuestas RFC 9457 en creación de recursos
  Dado que el Gestor de RRHH envía una solicitud POST al endpoint "/api/v1/recursos"
  Cuando suministra el nombre "<nombres>", DPI "<dpi>", departamento "<departamento>" y salario base
<salario_base>
  Entonces el endpoint retorna el código HTTP <status_http> con formato "<content_type>"

  Ejemplos:
    | nombres        | dpi           | departamento | salario_base | status_http | content_type             |
    | Elena Sofía    | 2540198270101 | Tecnología   | 18500.00     | 201         | application/json         |
    | Pedro Inválido | 12345         | Tecnología   | 10000.00     | 400         | application/problem+json |
    | Ana SalarioB   | 2999888770101 | Ventas       | 2500.00      | 400         | application/problem+json |
```

5. EXPERIENCIA DE USUARIO (UX/UI) Y MOBILE-FIRST (PASOS 8 Y 9)

El diseño centrado en el usuario (**ISO 9241-210**) incluye Customer Journey Maps, evaluación heurística de los 10 principios de Jakob Nielsen (**ISO 9241-11**), cumplimiento estricto de **WCAG 2.2 Nivel AA (Principios POUR: contraste 15.8:1, objetivos táctiles de 44×44px, navegación por teclado)** y arquitectura visual **Atomic Design** (Átomos, Moléculas, Organismos, Plantillas y Páginas en src/components/). El reporte completo se encuentra en docs/ui/reporte_ux_ui.pdf.

> 🎨 **Vínculo al Prototipo Interactivo Navegable en Figma:**
> https://www.figma.com/design/pdominguezr-ai/ERP-RRHH-Prototype

6. ESTRATEGIA DE CALIDAD Y PLAN SQA (PASO 10)

- **6.1 Plan SQA (IEEE 730-2014) y Shift-Left:** Documentado en docs/qa/plan_sqa_ieee730.pdf, diferenciando QA (prevención en proceso mediante DoR, INVEST y OpenAPI) de QC (detección mediante pruebas automatizadas).
- **6.2 Pirámide de Pruebas Automatizadas:** Implementada en tests/unit/gof_patterns.test.ts (pruebas unitarias de Strategy, Cache-Aside, Circuit Breaker, Observer, Gateway y RFC 9457) y tests/integration/api_endpoints.test.ts (pruebas de integración de los endpoints funcionales).
- **6.3 Clasificación y Gestión de Defectos (IEEE 1044-2009):** Documentada en docs/qa/reporte_defectos_ieee1044.pdf con severidad S1-S4, prioridad P1-P4 y flujo de resolución verificado.

7. MATRIZ DE TRAZABILIDAD INTEGRAL

Requerimiento (ISO 29148)	Historia de Usuario	Diagrama C4 (Nivel 2/3)	Endpoint OpenAPI	Código Backend (src/)	Escenario BDD	Caso de Prueba SQA
REQ-FN-001	HU-01	AuthContainer / Service	POST /api/v1/auth/login	src/controllers/AuthController.ts	autenticacion.feature	CP-01, CP-02 (ISO 29119)
REQ-FN-002	HU-02	ResourceCont / CacheAside	GET /api/v1/recursos	src/controllers/RecursoController.ts	gestion_recursos.feature	CP-04 (ISO 29119)
REQ-FN-004	HU-03	ResourceCont / PolyglotRepo	POST /api/v1/recursos	src/models/EmpleadoModel.ts	gestion_recursos.feature	CP-05, CP-06 (ISO 29119)
REQ-FN-006	HU-04	ResourceCont / CircuitBreaker	GET /api/v1/recursos/:id	src/controllers/RecursoController.ts	gestion_recursos.feature	CP-07 (ISO 29119)
REQ-FN-007	HU-05	PayrollEngine / Strategy	GET / POST /api/v1/empleados	src/services/gofPatterns.ts	gestion_principal.feature	CP-08 (ISO 29119)

8. CONCLUSIONES Y LECCIONES APRENDIDAS

1. **Valor de la Estandarización Internacional (ISO/IEEE):** La aplicación combinada de ISO 29148, IEEE 1016, ISO 25010, ISO 42010, ISO 29119, IEEE 730 e IEEE 1044 eliminó vacíos de especificación y garantizó trazabilidad de punta a punta.
2. **Beneficios de API-First, RFC 9457 y BDD:** Diseñar primero el contrato OpenAPI 3.0.3 junto con el formato Problem Details (application/problem+json) y los escenarios Gherkin permitió automatizar la validación funcional del backend sin ambigüedades.
3. **Impacto del Modelo C4 Completo y Persistencia Políglota:** Descomponer la arquitectura en los 4 niveles C4 y combinar PostgreSQL, MongoDB, Redis (Cache-Aside) y Neo4j optimizó tanto la integridad transaccional como la velocidad de respuesta.

9. REFERENCIAS

- ISO/IEC/IEEE. (2018). *ISO/IEC/IEEE 29148:2018 Systems and software engineering — Requirements engineering*.
- IEEE Computer Society. (2009). *IEEE Std 1016-2009 Standard for Information Technology—Systems Design—Software Design Descriptions*.
- ISO/IEC. (2011). *ISO/IEC 25010:2011 Systems and software Quality Requirements and Evaluation (SQuaRE)*.
- IETF. (2023). *RFC 9457: Problem Details for HTTP APIs* & *RFC 9110: HTTP Semantics*.
- W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*.
- IEEE Computer Society. (2014). *IEEE Std 730-2014 Software Quality Assurance Processes* & *IEEE Std 1044-2009 Classification for Software Anomalies*.